

Recruiter Login and Verification Flow

Backend contract for recruiter registration, email verification, sign in, organization review, protected hospital access, and password recovery.

Proposed API contract ? backend confirmation required

1. End-to-End Flow

The browser can guide navigation, but the backend must enforce recruiter role, email verification, organization approval, and organization ownership on every protected request.

2. Common API and Security Rules

ItemRequirement

Base URL | Frontend uses VITE_API_BASE_URL; endpoint paths below are relative to it. With the default value the login endpoint is /api/auth/login.

Format | JSON request and response bodies use application/json.

Roles | Canonical role is exactly recruiter for recruiter registration, verification, and login.

Email | Normalize and compare case-insensitively. Use the same account identity across registration, OTP, resend, and login.

Password | Hash server-side; never return it or log it. Current frontend requires 8-72 characters with uppercase, number, and symbol.

Verification code | Six numeric digits, single-use, short-lived (about 10 minutes), and rate-limited for resend and attempts.

Session | Return a short-lived access token. Prefer a Secure, HttpOnly refresh cookie; confirm refresh and revocation behavior with frontend.

Organization status | Return organizationVerificationStatus on the user as pending, approved, or rejected. Keep this field authoritative and update it when review status changes.

Authorization | Verify token, recruiter role, email verification, and organization approval on every recruiter resource endpoint. Never rely on React route guards for security.

3. Recruiter Registration

POST/auth/register

```
{
  "role": "recruiter",
  "fullName": "Rahul Menon",
  "email": "rahul@hospital.com",
  "phone": "+919876543210",
  "designation": "HR Manager",
  "organizationName": "MedCare Hospital",
  "organizationWebsite": "https://hospital.com",
  "city": "Chennai",
  "state": "Tamil Nadu",
  "password": "ExamplePassword@123",
  "termsAccepted": true
}
```

The UI collects a 10-digit phone number and currently prepends +91. The browser validates confirmation password but does not send it.

Backend: Validate required fields, persist recruiter and organization in a non-approved state, send registration OTP, and return 201 Created. Do not grant protected recruiter access at registration.

```
{
  "message": "Verification code sent",
  "userId": "user_123",
  "email": "rahul@hospital.com",
  "role": "recruiter",
  "emailVerified": false,
  "organizationVerificationStatus": "pending"
}
```

4. Email Verification and Resend

POST/auth/verify-code

```
{ "email": "rahul@hospital.com", "code": "482916", "purpose": "registration" }
```

On success, consume the code and mark email verified. Organization review remains independent; email verification must not approve the hospital.

```
{
  "verified": true,
  "purpose": "registration",
  "user": {
    "id": "user_123",
    "role": "recruiter",
    "email": "rahul@hospital.com",
    "emailVerified": true,
    "organizationVerificationStatus": "pending"
  }
}
```

POST/auth/resend-code

```
{ "email": "rahul@hospital.com", "purpose": "registration" }
```

Rate-limit by account and IP; return a generic response where appropriate to avoid account enumeration.

5. Recruiter Login and Routing Contract

POST/auth/login

```
{ "email": "rahul@hospital.com", "password": "ExamplePassword@123", "rememberMe": true }
```

rememberMe is a session-duration preference, not an account profile field. Return the recruiter role and organization state with the authenticated user.

```
{
  "accessToken": "eyJ...",
  "refreshToken": "refresh-token-value",
  "expiresIn": 3600,
  "user": {
    "id": "user_123",
    "role": "recruiter",
    "fullName": "Rahul Menon",
    "email": "rahul@hospital.com",
    "emailVerified": true,
    "organizationId": "org_456",
    "organizationName": "MedCare Hospital",
    "organizationVerificationStatus": "pending"
  }
}
```

Login response stateExpected frontend behaviorBackend requirement

Wrong account role | Reject recruiter login; do not store the session. | Return role accurately; frontend accepts only recruiter for recruiter login.

emailVerified: false | Request a registration code and send user to email verification. | Return verification state clearly; rate-limit resend.

Email verified + organization pending | Store the session and show Verification Pending; block recruiter resources. | Return organizationVerificationStatus: "pending".

Organization approved | Store session and enter recruiter dashboard/hospital profile. | Return organizationVerificationStatus: "approved".

Organization rejected | Do not grant dashboard access; show a clear support/resubmission path. | Return "rejected" and rejection guidance if policy allows.

Important implementation gap to resolve: the current login UI routes both pending and rejected states to the same pending screen. Decide the rejected-account UX and provide status/reason fields before production.

6. Approval and Protected Recruiter Resources

When MedHire approves an organization, update its status in the source of truth. On the recruiter's next login (or a future session-refresh/status endpoint), return the approved status. A stale browser session may continue to show pending until refreshed or reauthenticated; the backend must still enforce current approval on every resource request.

RequestRequired behavior

Pending/rejected recruiter calls recruiter data API | Return 403 with stable code ORGANIZATION_NOT_APPROVED (or a distinct rejected code). Return no protected data.

Approved recruiter calls recruiter data API | Authorize the recruiter and scope all reads/writes to their organization.

Missing, expired, or revoked token | Return 401; frontend must ask for login again.

Token belongs to candidate or another role | Return 403; never expose recruiter information.

Dashboard, candidate search, shortlist, purchased profiles, plans/credits, checkout, payment history/invoices, and hospital profile contracts are in Recruiter API Handoff.

7. Password Recovery for Recruiters

The existing recovery endpoints are shared across roles. Preserve account identity and role through the recovery UI.

POST/auth/forgot-password

```
{ "email": "rahul@hospital.com" }
```

Return a generic 202 Accepted whether the email exists or not.

POST/auth/verify-code

```
{ "email": "rahul@hospital.com", "code": "482916", "purpose": "password_reset" }
```

Return a short-lived, single-use reset token. Do not treat a reset code as email registration verification.

POST/auth/reset-password

```
{ "email": "rahul@hospital.com", "resetToken": "reset-token-value", "newPassword": "NewExamplePassword@123" }
```

Invalidate the reset token and revoke other sessions as required. The confirmation password remains frontend-only.

8. Validation and Error Contract

Frontend validation improves usability; the backend must independently repeat all security and business validation. Return field-level errors in this shared envelope:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Please correct the highlighted fields.",
    "fields": { "email": "Enter a valid email address." }
  }
}
```

StatusMeaning

400 / 422 | Malformed data or field/business validation failure.

401 | Invalid credentials, expired session, or invalid reset token.

403 | Wrong role, unverified email, or organization not approved.

409 | Email already registered or conflicting account state.

429 | Login, verification, or resend rate limit reached.

500 | Unexpected server error; do not include secrets or stack traces.

Current browser checks include required recruiter identity/organization/location fields, email format, ten-digit phone input, strong password, matching confirmation, accepted terms, required login password, and six-digit numeric verification code.

9. Frontend Flow and Files

StepRoute / fileCurrent behavior

Recruiter registration | /register/recruiter • src/pages/RegisterRecruiter.jsx | Calls recruiter registration wrapper; validates fields; moves to verification after API success.

Recruiter login | /login-role=recruiter • src/pages/Login.jsx | Checks response role/token; routes unverified users to OTP, approved users to dashboard, others to pending.

Email verification | /verify-email-role=recruiter • src/pages/EmailVerification.jsx | Verifies six-digit registration code and supports resend.

Pending review | /recruiter/verification-pending • src/pages/recruiter/RecruiterVerificationPending.jsx | Shows registration details and sign-out. Dashboard route guard returns unapproved users here.

Protected pages | /recruiter/* • src/components/recruiter/RequireRecruiterAccess.jsx | Client-side role/status gate. Backend authorization remains mandatory.

API wrappers | src/lib/recruiterAuthApi.js, src/lib/authApi.js | Recruiter role mapping and shared auth request/error handling.

10. Backend Completion Checklist

- Confirm endpoint paths, field names, error envelope, status codes, and API base URL.
- Implement recruiter registration that creates an unverified account and a pending organization record.
- Implement registration OTP delivery, expiry, one-time use, resend throttling, and attempt limits.
- Return role, email verification state, organization ID, organization name, and verification status from login.
- Define rejected-organization login and resubmission behavior.
- Implement secure access/refresh token lifecycle, session revocation, and logout/expiry behavior.
- Enforce role, email verification, current organization approval, and organization scoping server-side.
- Implement password recovery without disclosing account existence.
- Test pending, approved, rejected, unverified, wrong-role, expired-token, and rate-limited cases.
- Provide a staging base URL and test recruiter accounts for frontend end-to-end verification.

11. Current Integration Status

Prepared for the MedHire backend team. This is a proposed integration contract based on the current frontend. Confirm the API schema and security policy before implementation. Related documents: Authentication Backend Contract and Recruiter API Handoff.